

Guía de Referencia: Comandos Esenciales para Ciencia de Datos

Curso: Fundamentos de Ciencia de Datos
Propósito: Este documento sirve como guía de referencia rápida para el alumnado. Reúne las funciones y comandos más utilizados en las librerías fundamentales del ecosistema de Python para el análisis, visualización y modelado de datos.

1. NumPy (Cómputo Numérico y Matrices)

NumPy es la librería base para el cálculo científico en Python. Permite trabajar con vectores y matrices (arrays) de manera eficiente.

```
import numpy as np
```

Comando / Función	Descripción
<code>np.array([1, 2, 3])</code>	Crea un array de una dimensión (vector).
<code>np.zeros((3, 3)) / np.ones((2, 4))</code>	Crea matrices llenas de ceros o unos con las dimensiones especificadas.
<code>np.arange(start, stop, step)</code>	Genera un rango de valores numéricos espaciados uniformemente.
<code>np.linspace(0, 10, 5)</code>	Genera N números equidistantes entre dos límites (inclusive).
<code>array.shape</code>	Atributo que devuelve las dimensiones (filas, columnas) del array.
<code>array.reshape(filas, columnas)</code>	Cambia la forma de un array sin modificar sus datos.
<code>np.mean(arr) / np.median(arr) / np.std(arr)</code>	Funciones estadísticas básicas: media, mediana y desviación estándar.

2. Pandas (Manipulación y Análisis de Datos)

Pandas introduce las estructuras Series y DataFrame, ideales para trabajar con datos tabulares (filas y columnas).

```
import pandas as pd
```

Carga y Exploración Inicial

Comando	Descripción
<code>pd.read_csv('archivo.csv')</code>	Carga un archivo CSV en un DataFrame.
<code>df.head(n) / df.tail(n)</code>	Muestra las primeras o últimas n filas del dataset.
<code>df.info()</code>	Muestra un resumen del DataFrame (tipos de datos, memoria, nulos).
<code>df.describe()</code>	Genera estadísticas descriptivas de las columnas numéricas.
<code>df['columna'].value_counts()</code>	Cuenta los valores únicos en una columna categórica.

Indexación, Filtrado y Limpieza

Comando	Descripción
<code>df[['col1', 'col2']]</code>	Selecciona múltiples columnas.
<code>df.loc[fila, 'columna']</code>	Accede a filas y columnas por etiquetas/nombres.
<code>df.iloc[0:5, 0:3]</code>	Accede a filas y columnas por posiciones numéricas (indexación entera).
<code>df[df['edad'] > 30]</code>	Filtra filas basadas en una condición lógica.
<code>df.isnull().sum()</code>	Cuenta la cantidad de valores nulos por columna.

Comando	Descripción
<code>df.dropna()</code> / <code>df.fillna(valor)</code>	Elimina filas con nulos o reemplaza los nulos con un valor específico.
<code>df.drop(columns=['col1'])</code>	Elimina una o más columnas del DataFrame.

Agrupaciones y Operaciones

Comando	Descripción
<code>df.groupby('categoria')['num'].mean()</code>	Agrupar por una columna y calcular la media de otra.
<code>df.groupby('cat').agg(['mean', 'std'])</code>	Aplica múltiples funciones de agregación a los grupos.
<code>df.merge(df2, on='id', how='inner')</code>	Combina dos DataFrames usando una columna clave (similar a SQL JOIN).

3. Matplotlib & Seaborn (Visualización Estática)

Matplotlib proporciona el control total del gráfico, mientras que Seaborn ofrece una interfaz de alto nivel con estilos estéticos por defecto.

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

Comando	Librería	Descripción
<code>plt.plot(x, y)</code>	Matplotlib	Gráfico de líneas básico.
<code>plt.xlabel()</code> / <code>plt.ylabel()</code> / <code>plt.title()</code>	Matplotlib	Añade etiquetas a los ejes y el título principal.
<code>plt.show()</code>	Matplotlib	Muestra el gráfico en pantalla y limpia la memoria de la figura.
<code>sns.scatterplot(data=df,</code>	Seaborn	Gráfico de dispersión con distinción de colores por

Comando	Librería	Descripción
<code>x='x', y='y', hue='cat')</code>		categoría (hue).
<code>sns.histplot(data=df, x='col', kde=True)</code>	Seaborn	Histograma con curva de estimación de densidad (KDE).
<code>sns.boxplot(data=df, x='cat', y='num')</code>	Seaborn	Diagrama de caja y bigotes para analizar distribuciones y outliers.
<code>sns.heatmap(df.corr(), annot=True, cmap='coolwarm')</code>	Seaborn	Matriz de correlación gráfica con los valores anotados.

4. Plotly (Visualización Interactiva)

Plotly Express permite crear gráficos dinámicos con los que el usuario puede interactuar (hacer zoom, ver detalles al pasar el cursor).

`import plotly.express as px`

Comando	Descripción
<code>px.scatter(df, x='col_x', y='col_y', color='categoria')</code>	Gráfico de dispersión interactivo.
<code>px.line(df, x='fecha', y='valor', title='Tendencia')</code>	Gráfico de líneas temporal interactivo.
<code>px.bar(df, x='categoria', y='total', barmode='group')</code>	Gráfico de barras agrupadas o apiladas.
<code>fig.show()</code>	Renderiza e inicia la interactividad del gráfico en el navegador o Jupyter Notebook.

5. Scikit-Learn (Fundamentos de Machine Learning)

Scikit-Learn es la librería estándar para implementar modelos de Machine Learning clásico, abarcando preprocesamiento, entrenamiento y evaluación.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, classification_report
```

Flujo de Trabajo Esencial (Workflow)

Etapa	Código de Ejemplo	Descripción
1. División del Dataset	<code>X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)</code>	Divide los datos en entrenamiento (80%) y prueba (20%).
2. Escalado / Normalización	<code>scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test)</code>	Estandariza las variables numéricas (media=0, varianza=1). Usa <code>fit_transform</code> solo en entrenamiento.
3. Inicialización del Modelo	<code>model = LinearRegression()</code>	Crea la instancia del algoritmo elegido (ej. Regresión Lineal).
4. Entrenamiento	<code>model.fit(X_train_scaled, y_train)</code>	Ajusta el modelo utilizando los datos de entrenamiento.
5. Predicción	<code>predictions = model.predict(X_test_scaled)</code>	Genera las predicciones utilizando el conjunto de prueba.
6. Evaluación (Regresión)	<code>mse = mean_squared_error(y_test, predictions)</code>	Calcula el Error Cuadrático Medio.
7. Evaluación (Clasificación)	<code>print(classification_report(y_test, predictions))</code>	Muestra precisión, recall, F1-score y exactitud del modelo.

